Adaptive Page Replacement Based on Memory-Reference Locality

Yash Pradhan 24BCE0702 · Anjini Pandey 24BCE0714 · October 2026

Github Repository Link: GitHub link

Abstract

When a page fault occurs and every physical frame is occupied, the operating system must evict a resident page. Fixed rules such as FIFO, LRU, LFU and Clock make different assumptions about future references, so a rule that suits one access pattern can suit another poorly [1], [7], [8]. We implemented FIFO, LRU, Optimal, Clock and LFU, plus an adaptive policy that examines a rolling window of recent references and selects LRU, Clock or LFU using only references that have already occurred.

As we have tested seven page replacement policies using four workloads. Each workload was tested with 1,000, 5,000 and 20,000 page references, and with frame capacities of 3, 5 and 8. We had 252 configurations for 5 times totalling to 1260.

Adaptive W50, which uses a window of 50 references, gave the same number of page faults as LRU in 30 out of the 36 test conditions. In the other six, it reduced the number of faults by 1 to 6. It did not produce more faults than LRU in any condition. The results for Adaptive W100 were different in terms of switching: it matched LRU in all 36 conditions and never changed to another policy.

In testing LFU has performed better than the other policies in mean hit ratio, reaching 22.87%, while LRU achieved only 19.45%. In high locality workload with 5000 references and five frames difference is seen. LFU reached a hit ratio of 38.34%, whereas LRU and both adaptive versions reached 30.84%.

There was also a noticeable difference in execution time. The adaptive versions took about 4.6 to 5.4 times longer to run than LRU. Since the thresholds were chosen manually and were not tuned, the results suggest that the policy can switch according to recent page accesses, but this did not translate into a clear improvement in hit ratio for the workloads tested.

1. Introduction and Objectives

By definition, Virtual memory helps a process use more pages than what can fit in the physical memory. If a referenced page is not resident, a page fault occurs; if all frames are full, a replacement policy chooses a victim [1]. The best choice depends on how the program references memory, and the idea that programs show locality of reference underlies most practical policies [4]. A program may loop over a few pages for a long time and later scan many pages once, and a rule that protects the loop can behave badly during the scan.

Research question: Can an adaptive page-replacement policy select an appropriate strategy according to observed memory-reference locality, using only past references?

The objectives were to implement the five fixed policies; build an adaptive policy that chooses among them from a rolling window of past references while keeping resident pages on a switch; check each implementation against a textbook reference string; measure faults, hit ratio and execution time on synthetic workloads; and report where adaptation helps and where it does not.

2. Replacement Policies

Table 1 lists the policies as implemented in `src/`. Clock and LFU involve conventions that textbooks do not fix uniquely [7], [8], so the conventions used here are stated.

Policy	Rule used in this project
FIFO	Evicts the page resident longest.
LRU	Evicts the page whose most recent reference is furthest in the past [1].
Optimal (MIN)	Evicts the page whose next use is furthest in the future (or never) [2]. It needs future references, so it is an offline benchmark and is never used by an online policy.
Clock	Circular frames with a reference bit each; the hand starts at 0. A hit sets the bit to 1 and does not move the hand. On a fault with full frames, the hand clears bits equal to 1 as it advances and replaces the first page with bit 0; the new page gets bit 1 [1], [5].
LFU	Evicts the resident page with the lowest count. A count starts at 1 on loading and is lost on eviction; ties go to the earliest-loaded page (FIFO tie-break).

Table 1. Replacement policies as implemented.

Belady's anomaly is the effect, first shown for FIFO, in which more frames can cause more faults [3]; it is tested in Section 5.

3. The Adaptive Mechanism

`AdaptiveEngine` in `src/adaptive.py` keeps the resident set and a deque of the last W references (W = 50 or 100). The initial policy is LRU. At an evaluation step it computes, over the window,

$$U = \frac{\text{distinct pages in the window}}{W} \qquad CV = \frac{\sigma_f}{\mu_f}$$

where f is the list of counts of the distinct pages that appear in the window, μf is their mean and σf their population standard deviation (pages absent from the window are not counted as zeros). A large CV means a few pages dominate.

When it is checked. Only once the window is full and when step % max(10, W // 2) == 0, that is every 25 references for W = 50 and every 50 for W = 100.

Rules, tested in this order: (1) $U < 0.35$: LRU; (2) else $U \geq 0.85$: Clock; (3) else $CV \geq 1.0$ and $U < 0.70$: LFU; (4) otherwise LRU. The thresholds were set by hand; they were not learned and no tuning experiment was run. FIFO handover code exists, but no rule selects FIFO.

State handover. A switch keeps the resident pages and rebuilds only the new policy's bookkeeping from the window: for LRU, pages are ordered by last occurrence in the window (pages not in the window count as least recent); for LFU, each count is set to the page's count in the window (minimum 1); for Clock, all bits are set to 1 and the hand is reset to 0. History older than W references is therefore lost at a switch.

Causality. In reference() the engine chooses the policy before appending the current page to the history, so a decision depends only on earlier references, and the adaptive code never calls the Optimal implementation. One permanent test, test_causality_future_independence, checks this on a generated sequential trace of 1,000 references (W = 50, five frames): for k = 100, 437 and 900 it replaces every reference after step k with unrelated pages and confirms that the fault count, active policy and resident set at every step before k are unchanged. As we have tested, it covers one workload, one window size, one frame count and three cutoffs, the general argument rests on the code structure above.

4. Workloads, Setup and Validation

The traces come from `workload/workload_generator.py` (`random.Random(42)`) and are synthetic, not traces of real programs. High locality: 50 pages; each reference goes with probability 0.8 to one of 10 hot pages (0–9), otherwise to one of 40 cold pages (10–49). Sequential: a cyclic scan of 50 pages with a 2% chance per reference of jumping to a random page. Random: uniform over 50 pages.

The matrix is 4 workloads × 3 lengths × 3 frame capacities × 7 policies that gives 252 combinations and five timing repetitions and each of them gives 252 × 5 = 1,260 raw runs. Time is measured with time.perf_counter() around one simulation run; workload generation is excluded and Optimal's next-use preprocessing is included. These are Python simulation times, not kernel timings. Fault counts were identical in all five repetitions of every configuration, so repetitions affect only timings, and times below are means over the five. The recorded host was an Intel Core Ultra 7 155H, 15.46 GiB RAM, Windows 11 Home Single Language 64-bit (build 10.0.26200 at the run; later updated to 10.0.26300 without regenerating results), CPython 3.13.4, matplotlib 3.11.2 and numpy 2.5.3. The simulators use only the standard library.

Tests. The 29 unit tests (FIFO 5, LRU 4, Optimal 3, Clock 4, LFU 4, Adaptive 9) cover the textbook string, empty input, single frame, repeated pages, fault/hit accounting, determinism and the adaptive behaviours in Section 3. The Optimal test compares it with FIFO and LRU on a 20 reference string with the number of three frames. The scan-to-Clock test passes scan_threshold_U=0.8 (its input is 200 distinct pages, so

$U = 1.0$ either way). python -m unittest discover -s tests -v, re-run in a separate Linux environment (Python 3.13.16) while preparing this report, ran 29 tests and reported OK.

Textbook validation. On the string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 with three frames every policy matched its expected fault count (Table 2). The values of Clock and LFU depends upon the conventions in Table 1.

3 frames	FIFO	LRU	Optimal	Clock	LFU
Faults / hits	15 / 5	12 / 8	9 / 11	14 / 6	13 / 7
Hit ratio	25.00%	40.00%	55.00%	30.00%	35.00%
Expected faults (status)	15 (PASS)	12 (PASS)	9 (PASS)	14 (PASS)	13 (PASS)

Table 2. Textbook validation (results/validation_results.csv).

Belady's anomaly. FIFO on 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 gave 9 faults with three frames and 10 with four [3]. On the four loads N=1000 shows FIFO's faults did not increase from 3 to 8 frames. This shows only that the anomaly did not appear in this tested set. Also, no online policy had fewer faults than Optimal in any of the 36 conditions.

5. Results and Discussion

All values are stated from summary_results.csv and all 36 conditions is the mean over 4 workloads×3 lengths×3 frame capacities.

Policy	High locality	Sequential	Random	Looping	All 36 conditions	Mean time (ms)
FIFO	31.55%	0.98%	10.62%	33.25%	19.10%	2.392
LRU	32.81%	0.98%	10.77%	33.25%	19.45%	3.507
Clock	32.12%	0.98%	10.68%	33.25%	19.26%	9.219
LFU	40.80%	7.25%	10.19%	33.25%	22.87%	15.816
Adaptive W50	32.82%	0.99%	10.77%	33.25%	19.46%	18.971
Adaptive W100	32.81%	0.98%	10.77%	33.25%	19.45%	16.122
Optimal	55.92%	12.23%	32.03%	73.17%	43.34%	8.082

Table 3. Mean hit ratio by workload (9 length/frame settings each), over all 36 conditions, and mean execution time.

Optimal is highest, as an offline benchmark must be. Among online policies LFU has the highest overall mean (22.87%); LRU and both adaptive versions are within 0.01 points of each other (about 19.45%).

LFU is well ahead on the high-locality and sequential workloads and slightly behind LRU on the random one. All online policies are same on looping because they behave the same in every setting.

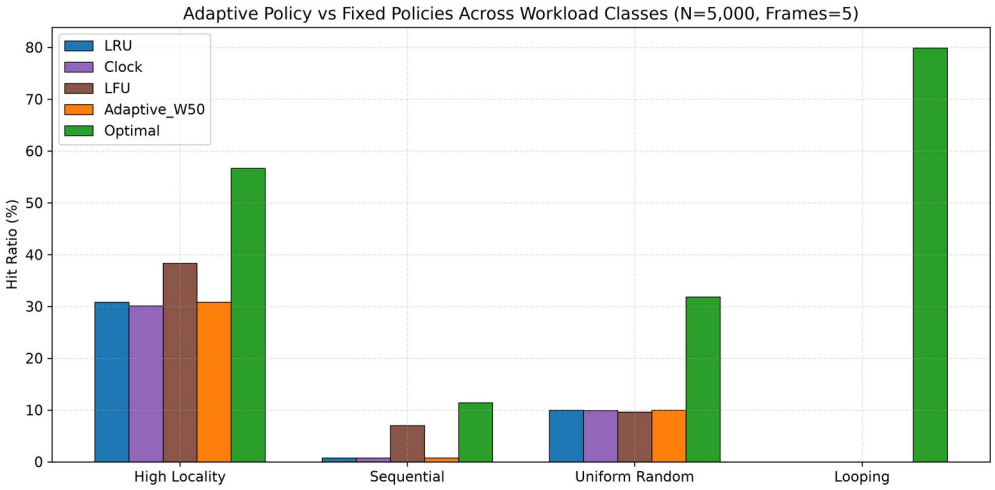


Figure 1. Hit ratio of LRU, Clock, LFU, Adaptive W50 and Optimal on each workload (N = 5,000, five frames). FIFO and Adaptive W100 are omitted here; see Table 4.

Workload	Frames	FIFO	LRU	Optimal	Clock	LFU	Adaptive W50	Adaptive W100
High locality	5	3,495	3,458	2,167	3,494	3,083	3,458	3,458
Sequential	5	4,964	4,962	4,431	4,964	4,650	4,962	4,962
Random	5	4,505	4,502	3,409	4,505	4,519	4,502	4,502
Looping	5	5,000	5,000	1,004	5,000	5,000	5,000	5,000
Looping	8	6	6	6	6	6	6	6

Table 4. Page faults for N = 5,000.

For high locality (N = 5,000, five frames) the hit ratios are LFU 38.34%, LRU 30.84% and both adaptive versions 30.84%, Clock 30.12%, FIFO 30.10%, Optimal 56.66% (Figure 1, left group of bars). A reason for LFU's lead is that with five frames and ten hot pages LFU keeps the highest count pages while LRU can lose hot pages after a run of cold references. In the sequential case every online policy except LFU has almost no hits, since the scan covers 50 pages and at most eight frames exist. On the random workload the policies are close (LRU 4,502 faults, LFU 4,519). The loop has six pages, with three or five frames every online policy faults on every reference (5,000 faults at five frames) while Optimal, which can look ahead, still gets hits (1,004 faults); with eight frames all pages stay resident after six compulsory faults, giving at least 99.40% hits at every length.

Adaptive versus LRU. W50 matched LRU's fault count in 30 of 36 conditions and had fewer in six, by 1 to 6 faults (high-locality and sequential workloads at N = 20,000, each at three frame capacities); for example 10,322 against 10,327 faults for high locality at eight frames, five faults in 20,000 references. W100 matched LRU in all 36. The gain is small.

Adaptive W50 switches	N = 1,000	N = 5,000	N = 20,000
High locality	0	0	2
Sequential	12	75	312
Random	0	0	0
Looping	0	0	0

Table 5. Policy switches of Adaptive W50 (same for 3, 5 and 8 frames).

Sequential workload. At N = 20,000 these are 156 changes to Clock and 156 back to LRU (at five frames, Clock was active for 11,075 references and LRU for 8,925). This is consistent with the 2% jumps, a jump makes pages repeat inside the window and can pull *U* below 0.85, returning the rule to LRU until a mostly distinct window returns. The hit ratio does not change from LRU (Figure 2, right).

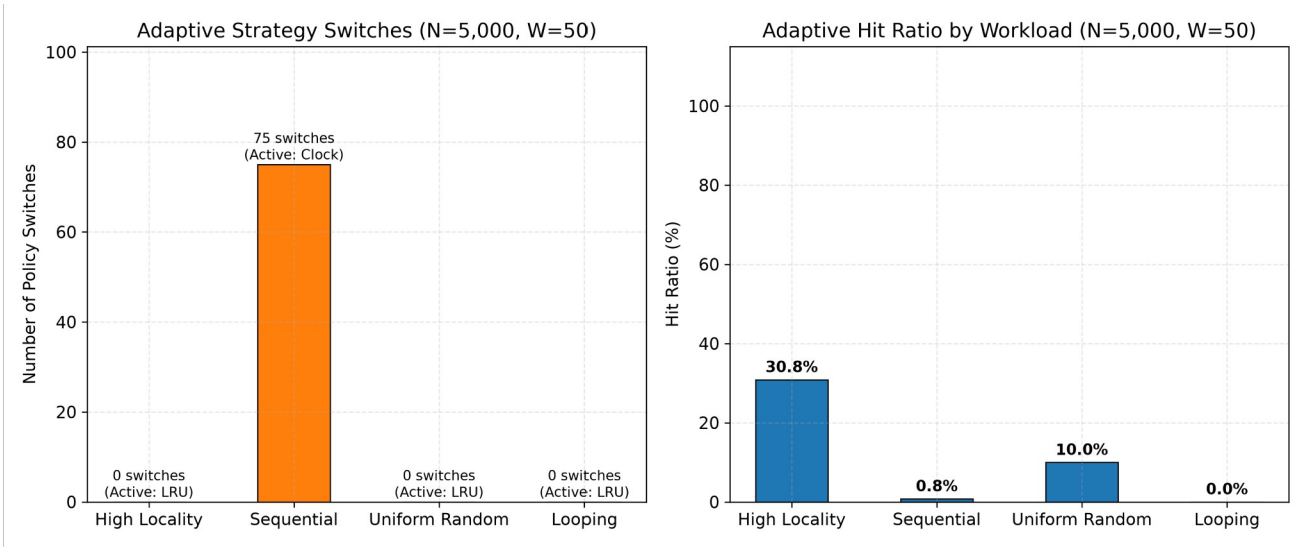


Figure 2. Adaptive W50, N = 5,000: policy switches (left) and hit ratio (right) per workload.

High locality. W50 did not switch at N = 1,000 or 5,000 and switched twice at N = 20,000 (to LFU and back; LFU was active for 25 references at five frames). Random workload and looping workloads are never switched. Recomputing the window metrics from the generated trace at the evaluation steps gives a maximum *CV* of about 0.92, 0.97 and 1.06 for N = 1,000, 5,000 and 20,000 (W50); only the last reaches the LFU threshold of 1.0. For W50, *U* lies between 0.24 and 0.54, so windows with $U \geq 0.35$ do reach the LFU test but fail on *CV*. That is why LFU, the best fixed policy here, was almost never selected.

Why W100 never switched. There are 50 different pages, so the ratio is always less than 1/2 when we look at 100 references.. On high locality the recomputed *U* at W100 never exceeded 0.33, so rule 1 applied every time and pre-empted the LFU rule even where *CV* reached about 1.15.

Execution time. Time is proportional to N in every case (Table 6). The adaptive versions compute window metrics at each evaluation; over the 36 conditions W50 took about 5.4 times and W100 about 4.6 times as long as LRU (Figure 3). These are Python simulation costs, not predictions of kernel behaviour.

N	FIFO	LRU	Optimal	Clock	LFU	Adaptive W50	Adaptive W100
1,000	0.272	0.482	0.995	1.122	1.989	2.287	1.791
5,000	1.297	2.022	4.598	5.301	8.874	10.806	9.027
20,000	5.608	8.016	18.654	21.234	36.584	43.821	37.549

Table 6. Mean execution time (ms) by reference-string length.

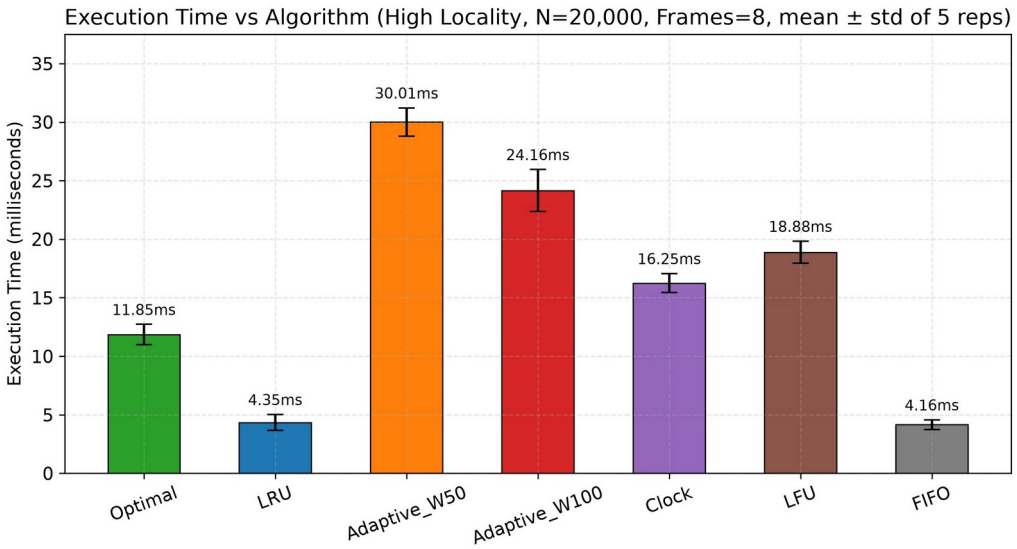


Figure 3. Mean execution time, high locality, N = 20,000, eight frames; error bars are the standard deviation over five repetitions.

6. Limitations and Future Work

The workloads are synthetic, stationary within each trace and small (50 pages; 6 for the loop); no trace with deliberate phase changes was tested, although that is what adaptive policies are meant for. The thresholds (0.35, 0.85, 1.0, 0.70) were set by hand and no tuning study was run, so any retuning, such as lowering the *CV* threshold so LFU is chosen on high locality, is future work with unknown effect. FIFO is never selected, and scan-resistant policies such as ARC [6] or MRU are not candidates. LRU order and LFU counts are rebuilt from the last W references only. Times come from one laptop, single-process Python, five repetitions, with interpreter overhead. The simulator does not model TLB effects, dirty-page write-back or storage latency.

7. Conclusion

The adaptive engine changed policy as the recent reference pattern changed and kept resident pages when it did so. On these workloads, however, switching gave almost no benefit: W50 matched LRU in 30 of 36 conditions and saved 1–6 faults in the rest, and W100 never switched. The rules explain this: *CV* on high locality stayed below the LFU threshold except in one case, and with W = 100 on 50 pages the Clock rule

cannot fire. The fixed LFU policy had the best online hit ratio on high-locality and sequential workloads, and the adaptive versions cost several times more simulation time than LRU. The evidence for the research question is therefore mixed: history-based selection works as designed but was not shown to choose better policies here, and its usefulness depends on thresholds that fit the workload.

8. Contributions and Complete Repository Link

Team. Yash Pradhan (24BCE0702) and Anjini Pandey (24BCE0714). The project's CONTRIBUTIONS.md assigns Yash the FIFO, LRU and Optimal implementations, their tests and the workload generator, and assigns Anjini Clock, LFU, the adaptive engine, the experiment runner and the analysis and plots; both students are responsible for the contents of this report.

Repository Link: GitHub link

Reproducibility. The repository holds the code, tests, traces, results and plots. Tables trace to results/summary_results.csv (252 rows), raw_results.csv (1,260 rows), validation_results.csv and belady_anomaly.csv; the environment is in report/experimental_setup.txt. Run the tests with python -m unittest discover -s tests -v. Re-running experiments/run_experiments.py regenerates the results; fault and switch counts are deterministic and re-simulating all 252 configurations reproduced every stored value, but timings depend on the machine.

References

1. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018 (Chapter 10, Virtual Memory).
2. L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal, vol. 5, no. 2, pp. 78–101, 1966.
3. L. A. Belady, R. A. Nelson and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," Communications of the ACM, vol. 12, no. 6, pp. 349–353, 1969.
4. P. J. Denning, "The working set model for program behavior," Communications of the ACM, vol. 11, no. 5, pp. 323–333, 1968.
5. F. J. Corbató, "A paging experiment with the Multics system," MIT Project MAC Report MAC-M384, 1968.
6. N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," Proceedings of USENIX FAST, 2003.
7. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2015.